

PULL REQUESTS AS ASSESSMENT INFRASTRUCTURE IN CDIO PROJECT COURSES

Helga Ingimundardóttir

Industrial Engineering, University of Iceland

ABSTRACT

Assessing collaboration, individual accountability, responsiveness to feedback, and reproducibility remains a persistent challenge in team-based, open-ended project courses. Final deliverables alone often conceal how work evolves, how decisions are made, and how feedback is addressed over time. This paper presents an implementation in which pull requests (PRs) serve as the central assessment artifact in project-based courses in Industrial Engineering, with GitHub Classroom used as stable collaboration and assessment infrastructure rather than as a programming topic or tooling tutorial. The implementation spans a two-course sequence. In a second-year mandatory course, students are explicitly onboarded to structured collaboration practices through short, skills-focused project cycles. In a subsequent advanced course, GitHub proficiency is assumed and assessment is organized around a single persistent team repository maintained across the semester. In this setting, PRs function as the primary unit of feedback, peer review, and evaluation. Students are required to author or co-author PRs, provide substantive peer reviews with multiple approvals, respond explicitly to instructor comments, and document reproducibility and handover within the repository. Unresolved feedback is carried forward across project cycles and revisited in capstone evaluation, enabling cumulative assessment rather than isolated grading of individual submissions. Early student resistance centered on perceived overhead and duplication of informal communication. Instructional responses focused on grading review quality, modeling effective feedback, and relocating collaboration onboarding earlier in the curriculum. Over time, PRs became more focused, review discussions more substantive, and documentation more consistent. Graduate reflections later characterized these practices as supportive of professional collaborative work. The paper focuses on design decisions, sequencing across courses, and trade-offs involved in using PRs as assessment infrastructure. It aims to provide transferable guidance for educators seeking to make collaboration, responsiveness, and reproducibility visible and assessable within team-based project courses.

KEYWORDS

Pull requests, GitHub Classroom, assessment infrastructure, team-based learning; professional competence, CDIO standards: 5, 7, 8, 11

INTRODUCTION AND MOTIVATION

Team-based projects pose persistent assessment challenges in CDIO-aligned curricula: instructors must evaluate not only technical outcomes but also how teams coordinate work, share responsibility, respond to feedback, and hand over results in reproducible, trustworthy ways. Prior CDIO work on logbook keeping illustrates that professional skills such as documentation and traceability require deliberate practice and structured assessment to persist beyond early courses (Junaid et al., 2018). Our approach extends this principle to collaboration artifacts: using pull requests as the locus for review, accountability, and reproducible handover.

Existing uses of GitHub Classroom in CDIO and computing-education contexts have focused primarily on *automated grading and assignment distribution* (Glassey, 2019), leveraging test frameworks and repository templates to streamline correctness checks rather than collaboration. While these implementations acknowledge the potential of richer GitHub features—such as pull requests—for structuring review and accountability, this remains largely aspirational or future work. In contrast, code review literature emphasizes that small, just-in-time reviews embedded in normal workflows are the most effective way to surface reasoning and improve reproducibility (Petre and Wilson, 2014). Similarly, practitioner guidance on scientific computing highlights documentation, version control, and transparent handover as essential for collaborative work (Wilson et al., 2017). This paper operationalizes those principles in a CDIO setting: treating pull requests as first-class assessment artifacts so that proposed changes, peer review, discussion, and revision are recorded together with the final integration decision, making collaboration and responsiveness visible and assessable (Petre and Wilson, 2014).

Throughout this paper, *pull requests (PRs)* are treated as principal assessment artifacts. A PR links a focused change set with an explicit description of intent, threaded review comments, and documented responses to feedback. Unlike commits or standalone reports, PRs capture how work evolves through critique and revision, and how responsibility is shared within a team. Relocating substantive communication from transient chat tools into Issues and pull request threads further supports traceability and accountability.

The motivation for this implementation is therefore not the adoption of a specific platform, but the construction of an assessment architecture that emphasizes collaboration, responsiveness to feedback, and reproducibility within project-based courses.

COURSE CONTEXT AND CURRICULAR PLACEMENT

The implementation spans two intentionally sequenced courses serving Industrial Engineering students, with regular enrollment from adjacent programs.

Information Engineering (IE) is a second-year mandatory course. Project work is smaller in scope and organized into short, skills-focused cycles that consolidate foundational competencies (e.g. structured analysis, data handling, visualization). GitHub collaboration practices—Issues for task tracking, branches for isolated work, and PRs for review and handover—are explicitly introduced and scaffolded. Each cycle uses a separate repository to reset expectations, isolate errors, and manage cognitive load for novices; the emphasis is skill development and guided practice over long-term continuity.

Business Intelligence (BI) is a third-year undergraduate/first-year master's elective with larger, data-intensive projects organized in thematic cycles that accumulate into a final deliverable. GitHub is the primary collaboration environment and proficiency is assumed rather than taught in parallel. Assessment is centered on *iterative, PR-based workflows* within a *single persistent team repository* maintained across the semester, enabling teams to revisit, resolve, and build upon feedback across successive PRs.

INITIAL IMPLEMENTATION AND IDENTIFIED LIMITATIONS

Historically, the BI course was delivered using one repository per cycle, with each cycle independent. PRs were used for review and feedback within cycles, but context and work traces were fragmented across repositories. To reduce student workload and enable cumulative iteration across the semester, BI later shifted to a single semester-long project culminating in a capstone project in which teams revisited and consolidated earlier work. An industry-engaged framing of this redesign is described in Ingimundardóttir (2026); the present paper focuses on the assessment architecture enabled by PRs and repository continuity.

Across these iterations, several limitations became apparent:

1. **Feedback discontinuity:** when repositories closed at cycle boundaries, instructor comments became effectively *non-binding*.
2. **Instructor overhead:** collaboration and assessment rules had to be repeatedly configured for each repository.
3. **Fragmented traceability:** diffs, discussions, and documentation were split across repositories, complicating auditing of how feedback was addressed over time.

A curricular refinement followed: GitHub onboarding moved to the second-year IE course, where collaboration workflows are explicitly taught and practiced. With proficiency now assumed in BI, onboarding overhead is minimal. BI therefore adopts a single persistent team repository with sustained PR continuity so that feedback, decisions, and documentation remain visible and actionable throughout the semester. This sequencing and explicit allocation of responsibilities across courses reflects CDIO program design practices that connect goals to courses via integrated program descriptions and planned learning sequences (Malmqvist et al., 2006).

DESIGN RATIONALE: PULL REQUESTS AS ASSESSMENT INFRASTRUCTURE

This section articulates the rationale for treating PRs as assessment infrastructure rather than as a procedural workflow layered on top of project work. The design choices described here are guided by a single objective: to make collaboration, responsiveness to feedback, and responsibility for shared artifacts visible and assessable through normal project activity. Instead of introducing additional deliverables or parallel assessment channels, the assessment logic is embedded in existing collaboration traces, allowing evaluation to focus on how work evolves through review, revision, and integration over time.

Infrastructure Context: GitHub Classroom

GitHub Classroom provisions standardized team repositories from a common template and centralizes instructor interactions within repository artifacts (issues, reviews, and feedback pull requests; see documentation for workflow details in GitHub Inc., 2026a, 2026b). In CDIO practice, Classroom has been used to organize assignment distribution and feedback within repositories rather than through separate submission channels (Bennedsen et al., 2022). More broadly, computing-education tooling surveys document widespread use of Git/GitHub for assignment management, batch cloning, and feedback through issues/PRs, underscoring the feasibility of treating the repository as the primary assessment locus (Glassey, 2019).

Why PRs Rather Than Commits or Reports

We define PRs as the primary assessment artifact because they bind *technical change* to *collaborative process*. A PR couples a focused change set with rationale and scope, line-level review and threaded discussion, explicit responses to comments, and a recorded decision to merge or revise. This locates assessment in the same place where communication and accountability naturally occur during code review (Petre and Wilson, 2014), and reflects established classroom practice in using GitHub artifacts for assignment review and feedback (Glassey, 2019). Commits rarely expose reasoning; standalone reports detach explanation from the evolving artifact. PRs therefore make intent, critique, revision, and decision visible and assessable within normal project activity.

ASSESSMENT ARCHITECTURE AND IMPLEMENTATION PRACTICES

This section details the practical mechanisms through which the assessment architecture is enacted across courses, including assessable behaviors, efficiency gains from repository-based workflows, curricular sequencing, and key design trade-offs.

Assessable Behaviors and Required Practices

To make targeted behaviors explicitly assessable and consistently reviewable, the courses require: (i) focused PRs with clear descriptions (and linked Issues when relevant); (ii) at least two *substantive* peer reviews before merge; (iii) explicit written responses to peer and instructor comments (no silent fixes or off-platform clarification); (iv) merge only after approvals, with any deferrals documented as follow-up Issues; (v) README and run-instruction updates, plus concise experiment summaries, when execution or interpretation changes; and (vi) visible authorship/co-authorship and review participation (visibility over volume of code).

These requirements draw on industry practice, where pull requests function as the locus for communication, accountability, and reproducible handover. They align with practitioner guidance that situates rationale, critique, and merge decisions within review threads (Petre and Wilson, 2014), as well as “good enough” scientific computing norms emphasizing documentation-first workflows and transparent change tracking (Wilson et al., 2017).

Assessment Efficiency and Reproducibility

Repositories and PRs eliminate ad hoc packaging by keeping all submission files *in situ* within a standard project structure. Code is expected to run in that structure, with shared datasets explicitly included or referenced. This reduces non-pedagogical overhead and shifts evaluation toward method, documentation, and responsiveness. The repository serves as a stable execution and review environment, improving reproducibility and assessment consistency.

Sequencing Across Courses

The prerequisite course introduces and rehearses PR conventions in short, skills-focused cycles (multi-repository) to manage novice load. The advanced course assumes fluency and leverages a *single persistent team repository* for cumulative iteration, traceability, and efficient capstone evaluation, keeping attention on collaboration quality rather than tooling.

Design Trade-offs and Scope

The design trades early overhead (learning PR etiquette, writing clearer descriptions, conducting substantive reviews) for gains in transparency, fairness, and both learning-focused and grading efficiency. PR-centric workflows align assessment with widely used version-control practices, supporting students' preparedness for professional teamwork without requiring external industry involvement. The goal is assessable evidence of collaboration and development grounded in observable project artifacts.

ASSESSMENT FOCUS: REPRODUCIBILITY, HANDOVER, AND RESPONSIVENESS

Assessment extends beyond technical correctness to emphasize *reproducibility*, *handover*, and *responsiveness to feedback*. This emphasis parallels findings from CDIO implementations that treat logbook keeping as a professional skill requiring explicit assessment and feedback to sustain good practice (Junaid et al., 2018).

Reproducibility is assessed through documentation maintained within the repository, with particular emphasis on an evolving README, clear run instructions, and concise summaries of analytical steps or experiments—consistent with practitioner guidance favoring documentation-first, shared workflow norms (Wilson et al., 2017). Handover is operationalized through PRs: clear intent and scope, linked issues, and explicit responses to comments make integration decisions and rationale traceable (Petre and Wilson, 2014). Responsiveness to feedback is treated as a central assessed behavior: students engage directly in PR discussions to clarify assumptions, address questions, and revise work where appropriate.

Together, they shift assessment from isolated task completion toward responsibility for the coherence, continuity, and robustness of shared project work, consistent with *constructive alignment* practices that emphasizes observable evidence of attainment (Biggs and Tang, 2015).

Reproducibility as an Assessable Outcome

Reproducibility is assessed through documentation maintained within the repository, with particular emphasis on a clear and evolving README. Contributions that affect execution or inter-

pretation are expected to include concise run instructions, descriptions of data dependencies, and summaries of analytical steps or experiments. Because documentation is reviewed as part of pull requests, reproducibility is treated as an integral component of each change rather than as an auxiliary deliverable. This allows peers and instructors to verify results within the shared project context and establishes reproducibility as a routine responsibility rather than a final-stage check.

Handover Through Pull Requests

Handover is operationalized through pull requests. Each pull request is expected to communicate intent and scope clearly: what was changed, why it was done, and how it relates to the broader project. By requiring this explanation at the moment of integration, the pull request functions as an explicit handover between team members rather than a private implementation step followed by implicit acceptance. Clear descriptions and linked issues support shared understanding and reduce the risk of knowledge becoming siloed within the team.

Responsiveness and Shared Understanding

Responsiveness to feedback is a central assessment criterion. Students are expected to engage explicitly with peer and instructor comments in pull request discussions by answering questions, clarifying assumptions, and revising work where appropriate. Understanding others' contributions is therefore not optional. Students are encouraged to ask questions directly in pull request threads when aspects of a contribution are unclear; such questions are treated as positive evidence of engagement and shared responsibility rather than as signs of deficiency. Assessment rewards visible efforts to surface uncertainties and to resolve misunderstandings within the team.

Linking Team Artifacts to Individual Learning Outcomes

Taken together, these assessment focuses reinforce collective accountability and system-level responsibility. Students are evaluated on their ability to make contributions that others can understand, review, reproduce, and extend, reflecting the expectation that responsibility in project work extends beyond individual task ownership to the integrity and maintainability of the evolving system.

Within team-based assessment, these criteria also support *individual* attainment of course-level intended learning outcomes. Assessment is grounded in observable evidence that can be attributed to individual students within shared project artifacts. Pull request authorship and co-authorship, the quality of peer review, and explicit responsiveness to feedback provide concrete evidence of each student's engagement with and demonstration of the competencies specified in the course curriculum, even when final deliverables are produced collaboratively.

FEEDBACK CARRY-FORWARD AND CAPSTONE RE-EVALUATION

Project work is organized in thematic cycles that accumulate into a final capstone deliverable. Instructor feedback is provided primarily through pull requests during each cycle, focusing on methodological soundness, documentation, and clarity of intent. Crucially, feedback is not closed at cycle boundaries.

Unresolved pull request comments and questions explicitly *carry forward* into the capstone phase. At final submission, evaluation checks whether previously raised issues have been addressed. If a concern remains unchanged, the corresponding assessment outcome remains unchanged as well. This policy makes feedback binding without requiring resubmission of earlier work and signals that responsibility for addressing feedback persists across the semester. The feasibility of using issues and PRs as persistent feedback artifacts aligns with broader observations of GitHub workflows in computing education (Glassey, 2019).

This approach enables efficient summative assessment. Because earlier reviews document expected revisions and rationale, instructors can focus the capstone evaluation on resolution of identified issues, using pull request diffs to verify changes rather than re-reviewing entire codebases or full reports. Assessment thus builds on prior formative dialogue instead of resetting at the capstone stage.

The intent of carry-forward evaluation is developmental. Early iterations may include exploration and partial solutions, provided limitations are acknowledged and tracked. The capstone rewards teams that revisit unresolved issues, integrate feedback, and deliver coherent, reproducible outcomes. In this way, assessment reinforces iterative improvement and shared accountability while preserving space for learning during earlier cycles.

STUDENT RESPONSE AND ADAPTIVE REFINEMENT

Initial student response highlighted perceived overhead and duplication of existing team communication practices. Common objections included: (i) “we already discuss work on Messenger”; (ii) pull requests were perceived as extra busywork; and (iii) review requirements were seen as slowing progress. Resistance was strongest early on, particularly around relocating discussion into Issues and pull request threads and writing explicit responses to comments.

Instructional adjustments focused on behavior rather than tool mastery. Review quality was explicitly assessed, making superficial approvals visible in grading. Instructors modeled effective review comments, such as clarifying assumptions, requesting justification, and pointing to reproducibility checks. In addition, GitHub onboarding was relocated earlier in the curriculum so that later courses could assume fluency rather than develop skills in parallel with complex project work. Across successive offerings, observable practices shifted: pull requests became smaller and better scoped, documentation quality improved, and review discussions showed more substantive questioning and revision.

Graduate reflections later characterized the approach as valuable for professional practice. Alumni reported that visible review processes, documentation habits, and explicit handover supported effective collaboration in workplace teams. While these reflections are qualitative and context-specific, they informed continued use of the pull-request-centric design and reinforced the decision to assess collaboration through persistent, observable project artifacts rather than end-stage self-report.

ALIGNMENT WITH CDIO STANDARDS

The implementation aligns primarily with CDIO Standards 7, 8, and 11, with partial alignment to Standard 5 (Malmqvist et al., 2020, 2022). Rather than adding separate instructional components, the approach embeds collaboration, assessment, and professional practice directly into ongoing project work. Integrated Learning Experiences (Standard 7) are realized through pull-request-centered workflows that require students to articulate intent, review peers' contributions, document assumptions, and coordinate handover, integrating technical and interpersonal competencies within authentic development activity. Active Learning (Standard 8) emerges through iterative contribution cycles in which students propose changes, receive critique, revise work, and integrate peer and instructor feedback in situ through PR discussion threads. Learning Assessment (Standard 11) is addressed through observable artifacts—authorship, review depth, responsiveness to comments, and the reproducibility of documented workflows—with unresolved feedback carrying forward across cycles to support cumulative evaluation. Partial alignment with Design–Implement Experiences (Standard 5) is evident in the use of persistent repositories and pull-request iteration to support sustained design and implementation in semester-long projects, with scaled-down versions used earlier in the curriculum. Together, these elements position PR-driven collaboration as both a technical practice and a pedagogical mechanism for enacting CDIO principles within routine project activity.

TRANSFERABLE DESIGN PRINCIPLES

The following principles are intended to be reusable across CDIO-aligned programs, independent of any specific platform.

1. **Treat collaboration artifacts as assessment evidence.** Make PRs (or equivalent review units) the basis for grading so that intent, critique, revision, and merge decisions are visible.
2. **Sequence tool learning before complex project work.** Teach collaboration workflows in a lower-stakes course; assume proficiency in advanced, open-ended projects.
3. **Prefer persistent artifacts over fragmented tasks.** Use a single team repository for sustained projects to accumulate feedback, documentation, and decisions and to support traceability.
4. **Grade review quality and responsiveness.** Reward substantive peer critique and explicit responses; penalize cursory approvals and off-platform clarification.
5. **Carry feedback forward.** Treat unresolved comments as work-to-be-done and re-evaluate them at integrative milestones rather than resetting assessment.

CONCLUSIONS

This paper presents a CDIO implementation in which pull requests function as assessment infrastructure for team-based project courses. By anchoring evaluation in pull request authorship and review, explicit responsiveness to feedback, and reproducible handover, the design makes collaboration, decision-making, and individual participation observable within normal project activity. Persistent team repositories support continuity across project cycles and enable efficient capstone re-evaluation without re-reviewing entire codebases or reports.

The contribution lies not in the tool alone, but in the assessment architecture: pull requests as the unit of feedback and grading, peer review as an assessed responsibility, and systematic carry-forward of unresolved feedback across the semester. While GitHub Classroom is commonly used for assignment distribution and automated grading, there is limited prior work on using pull requests themselves as the primary assessment artifact; the contribution of this paper lies in this assessment architecture rather than in the underlying tool. Sequencing GitHub onboarding in an earlier course allows advanced project courses to use these mechanisms naturally in complex, data-intensive work. Future refinements will focus on lightweight review rubrics, clearer PR templates for documentation and reproducibility, and continued calibration of expectations across mixed undergraduate-graduate cohorts.

ACKNOWLEDGEMENTS

The author received no financial support for this work. The author thanks the students who participated in these courses for their sustained engagement and for trusting a demanding collaboration and assessment approach. While adopting professional PR workflows requires substantial effort, the students' commitment demonstrated the long-term value of making collaboration explicit, visible, and accountable.

REFERENCES

- Bennedsen, J., Böttjer, T., & Tola, D. (2022). Using github classroom in teaching programming. *Proceedings of the 18th International CDIO Conference*.
- Biggs, J., & Tang, C. (2015). Constructive alignment: An outcomes-based approach to teaching anatomy. In L. K. Chan & W. Pawlina (Eds.), *Teaching anatomy: A practical guide* (pp. 31–38). Springer International Publishing. https://doi.org/10.1007/978-3-319-08930-0_4
- GitHub Inc. (2026a). Collaborating with pull requests [Accessed: 2026-01-13]. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests>
- GitHub Inc. (2026b). Leave feedback with pull requests [Accessed: 2026-01-13]. <https://docs.github.com/en/education/manage-coursework-with-github-classroom/teach-with-github-classroom/leave-feedback-with-pull-requests>
- Glasse, R. (2019). Adopting git/github within teaching: A survey of tool support. *Proceedings of the ACM Conference on Global Computing Education*, 143–149. <https://doi.org/10.1145/3300115.3309518>
- Ingimundardóttir, H. (2026). *Designing authentic industry-engaged assessment for professional competence in business intelligence* [Manuscript submitted to the 22nd International CDIO Conference, Liverpool, United Kingdom].
- Junaid, S., Gorman, P. C., & Leslie, L. J. (2018). Developing logbook keeping as a professional skill through CDIO projects. *Proceedings of the 14th International CDIO Conference*.
- Malmqvist, J., Edström, K., & Rosén, A. (2020). CDIO standards 3.0—updates to the core CDIO standards. *Proceedings of the 16th International CDIO Conference*, 60–76.
- Malmqvist, J., Lundqvist, U., Rosén, A., & Edström, K. (2022). The CDIO syllabus 3.0—an updated statement of goals. *Proceedings of the 18th International CDIO Conference*.
- Malmqvist, J., Östlund, S., & Edström, K. (2006). Integrated program descriptions—a tool for communicating goals and design of CDIO programs. *Proceedings of the 2nd International CDIO*

Conference. <https://cdio.org/knowledge-library/documents/integrated-program-descriptions-tool-communicating-goals-and-design>

Petre, M., & Wilson, G. (2014). Code review for and by scientists. In D. Katz (Ed.), *Proceedings of WSSSPE 2014*. <https://arxiv.org/pdf/1407.5648>

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>

BIOGRAPHICAL INFORMATION

Helga Ingimundardóttir is an Assistant Professor of Industrial Engineering at the University of Iceland. Since 2023, her teaching and research have focused on optimization, mathematical modeling, data science, artificial intelligence, and industry-engaged learning in data-intensive contexts. She is a member of the Teaching Academy of the public universities in Iceland (2024). Helga brings extensive industry experience across research and applied AI: as a Research Scientist at *deCODE Genetics*, she designed long-read sequencing analysis pipelines and co-authored papers in *Nature Genetics* and *Genome Biology*; as a Data Scientist at *CCP Games*, she helped develop real-time recommendation services for *EVE Online*; as Head of AI Research at *Travelshift*, she led large-scale trip-planning optimization; and earlier, as a Computational Engineer in R&D at *Valka* (now JBT Marel), she developed X-ray-based optimization methods for automated fish processing. She holds a PhD and MSc in Computational Engineering and a BSc in Mathematics from the University of Iceland, as well as a graduate diploma in Teaching Studies for Higher Education.

Corresponding author

Helga Ingimundardóttir
University of Iceland
School of Engineering and Natural Sciences
Faculty of Industrial Engineering, Mechanical
Engineering and Computer Science
VR-II, Hjarðarhagi 6
107 Reykjavík, ICELAND
helgaingim@hi.is



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International License